Expandir tudo | Recolher tudo

Estado 236

Empacotando uma Aplicação com ARDUINO IDE 263

O Django Framework 279

Modelos 309

Formulários e Templates 337

Criando Telas de Cadastro com o Django Admin 371

Construindo a Aplicação de Uma Loja Virtual com o Django 377

Apêndice 1: Instalação de Ferramentas no Windows 425

Apêndice 2: Instalação de Ferramentas no Linux 430

Referências Bibliográficas 433

14

O DJANGO FRAMEWORK

Neste capítulo, você terá seu primeiro contato com o Django — conhecido como "o framework para perfeccionistas com prazos a cumprir". Um ***framework*** é um conjunto de classes implementadas em uma determinada linguagem de programação, que serve para facilitar a criação de aplicações. O termo muitas vezes serve para designar um conjunto de arquivos de bibliotecas, códigos-fonte ou compilados e até mesmo recursos, tais como ícones e folhas de estilo, que podem ser reutilizados.

Em particular na sua página online www.djangoproject.com, o Django Framework promete levar suas aplicações web de um conceito à produção em uma questão de horas. Pessoalmente, acho essa promessa um tanto exagerada, a não ser que sua aplicação seja muito simples, mas serve para ilustrar o potencial da ferramenta para aumentar a produtividade do seu time de desenvolvimento.

Por que mais um framework? E por que Django?

Se você trabalhou na área de informática no período de 2000 a 2005, provavelmente lembra dos "frameworks da semana". Naquela época, a linguagem Java reinava, quase absoluta, no desenvolvimento web e uma afirmação que ouvi muito naquela época foi: "programar para a web em Java sem um framework, não dá!" Acho que, de tanto ouvirem isso, as pessoas se animaram a desenvolver os próprios frameworks, de modo que quase toda semana um novo produto aparecia prometendo ser "a solução de todos os problemas para seu desenvolvimento web". O tempo passava e as pessoas começavam a se dar conta de que, por melhor que fosse a ferramenta, sempre havia uma curva de aprendizado e esta era, raramente, suave. Mesmo assim, havia uma lição a ser extraída dessa situação: desenvolver software para a web é uma tarefa grande e complexa e, sem ajuda, ela se torna muito pesada.

Então vem a outra pergunta: se você vai usar um framework, por que justamente Django?

Primeiro, por ser um produto maduro: ele já está no mercado há mais de uma década e é utilizado por grandes empresas, tais como Instagram, Spotify, Youtube, Dropbox, só para citar alguns nomes.

Outra coisa que realmente diferencia o Django da "concorrência" é sua abordagem pragmática: você consegue implementar ***muita coisa*** em um tempo relativamente curto. Ele é conhecido com um framework que vem com "pilhas inclusas" (lembra, na sua infância, como era frustrante ganhar um brinquedo e descobrir que ele não vinha com as pilhas e que no Natal não havia lojas abertas para comprá-las? Então, os fabricantes de brinquedos começaram a pôr pilhas na embalagem e anunciar nas caixas de seus produtos como mais uma vantagem: pilhas inclusas). No caso do Django, "as pilhas inclusas" referem-se a um monte de implementações de processos comuns, porém complexos, que são fornecidos junto com o framework para serem utilizados de maneira simples (seguindo o "Zen do Python": "simples é melhor que complexo"). Essas "pilhas" estão localizadas no grupo de pacotes conhecido como *"**contrib packages**"*:

- ***admin***: Permite administrar sua aplicação, até mesmo fornecendo uma interface simples para criação, atualização, exclusão e consulta de objetos — que pode ser reaproveitada pela sua aplicação.
- ***auth***: Provê serviços de segurança como autenticação (validação de login e senha, por exemplo) e autorização (permite definir quais grupos de usuários podem ou não usar certas partes do sistema).
- ***contenttypes***: Permite gerenciar os modelos (classes de negócio) do Django com simplicidade.
- ***flatpages***: Fornece gerenciamento para páginas especiais, para as quais você não precisa desenvolver uma aplicação personalizada; páginas simples, muitas vezes com conteúdo estático como "Sobre a minha loja virtual" ou "Política de trocas".
- ***gis***: Adiciona capacidade de lidar com informações geoespaciais ao Django (e.g., interagir com o "Google Maps").
- ***humanize***: Acrescentar filtros aos templates (modelos de páginas) para melhorar a legibilidade das informações.

Expandir tudo | Recolher tudo

Escola 263
Preparando uma Aplicação com AMQP e JP 279
O Django Framework 309
Modelos 337
Formulários e Templates 371
Criando Telas de Cadastro com o Django Admin 377
Construindo a Aplicação de Loja Virtual com o Django 425
Apêndice 1: Instalação de Ferramentas no Windows 430
Apêndice 2: Instalação de Ferramentas no Linux 433
Referências Bibliográficas

- ***flatpages***: Fornece gerenciamento para páginas especiais, para as quais você não precisa desenvolver uma aplicação personalizada, páginas simples, muitas vezes com conteúdo estático como "Sobre a minha loja virtual" ou "Política de trocas".
- ***gis***: Adiciona capacidade de lidar com informações geoespaciais ao Django (e.g., interagir com o "Google Maps").
- ***humanize***: Acrescentar filtros aos templates (modelos de páginas) para melhorar a legibilidade das informações.
- ***messages***: Provê gerenciamento de mensagens baseadas em cookies e dados armazenados na sessão.
- ***postgres***: Fornece integração com o SGBD PostgreSQL.
- ***redirects***: Controla redirecionamento de chamadas entre os métodos do protocolo HTTP.
- ***sites***: Permite operar vários sites a partir de uma mesma instalação.
- ***sitemaps***: Gera, automaticamente, arquivos XML que refletem um mapa do seu site.
- ***staticfiles***: Fornece arquivos estáticos a partir de suas aplicações Django.
- ***syndication***: Gera feeds RSS e Atom, que são usados para enviar atualizações de novidades do seu site para usuários registrados.

Se achou que muitas dessas funcionalidades lembram o site de um jornal ou revista, você está absolutamente correto! Django surgiu, originalmente, como um gerenciador de conteúdo especializado em sites de notícias, mas cresceu e mostrou potencial para muito mais, como você verá neste capítulo.

Ele também provê versões estáveis e suportadas por longo período (as chamadas *LTS — Long Term Support*), que, tipicamente, são suportadas por três anos ou mais. Este livro foi escrito usando a versão 2.2 LTS, lançada em 01/04/2019, e que terá suporte até o segundo semestre de 2023.

Além de uma miríade de pacotes desenvolvidos por terceiros e, em grande parte, gratuitos e de código aberto.

Princípios do framework Django

Há uma impressionante variedade de tecnologias com as quais você pode criar uma aplicação web. Se já desenvolveu para essa plataforma antes, sabe que o que varia bastante é o tempo de desenvolvimento, dependendo da tecnologia escolhida. Esse tempo pode ser reduzido, usando o processo encorajado pelo framework. Não vou incorrer no que se chama em Engenharia de Software de "Síndrome da Bala de Prata" — sei que toda tecnologia tem seus prós e contras e Django não resolverá todos os seus problemas da noite para o dia, mas, comparado com outras tecnologias com as quais trabalhei, a impressão que tenho é de que ele é uma das mais produtivas.

Django trabalha sobre alguns princípios que norteiam o seu uso:

1. ***DRY*** ("*Don't Repeat Yourself*" [Não se repita ou Não fique se repetindo, em tradução livre]): É comum, no desenvolvimento para a web, ocorrer redundância de códigos entre as diversas camadas da aplicação; interface com o usuário em HTML; regras de negócio em uma ou (geralmente) várias classes no servidor; código que acessa e atualiza informações no seu SGBD; scripts de geração das tabelas; uma API REST para comunicação com outras interfaces com o usuário; e, muitas vezes, cada camada dessas possui códigos quase idênticos, resultando em um monte de ciclos de "copiar e colar", o que não é reúso, e sim ***redundância***.

Por exemplo, criar uma simples tela de manutenção de produtos, em que você cadastraria produtos, alteraria seus dados, desabilitaria um produto para que não fosse mais vendido ou apenas consultaria os dados de um ou mais produtos, tipicamente, envolve:

- Criar um script de banco de dados para armazenar as informações dos produtos. Você teria que criar um script (na maioria das vezes, em linguagem SQL) para definir o nome da tabela, os campos e seus tipos de dados, possíveis índices para acelerar a pesquisa e restrições como, por exemplo, não pode haver dois produtos com o mesmo nome.
- Criar classes que fazem o gerenciamento dos acessos ao banco de dados. Mesmo que você use um mecanismo de persistência para facilitar essa tarefa, ainda existem muitas configurações necessárias para evitar possíveis problemas.
- Criar classes que implementam regras de negócio. Essas normalmente são as mais trabalhosas e difíceis, quando não impossíveis, necessárias para implementar os requisitos independentes do restante das classes (ou seja, não

Expandir tudo | Recolher tudo

Estado 255
Empacotando uma Aplicação com AMQP e JSP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

dados, possíveis índices para acelerar a pesquisa e restrições como, por exemplo, não pode haver dois produtos com o mesmo nome.

- Criar classes que fazem o gerenciamento dos acessos ao banco de dados. Mesmo que você use um mecanismo de persistência para facilitar essa tarefa, ainda existem muitas configurações necessárias para evitar possíveis problemas.
- Criar classes que implementam regras de negócio. Essas costumam ser as mais trabalhosas e é difícil, quando não impossível, conseguir fazê-las totalmente independentes do restante das classes (na realidade, por sua natureza, elas ***costumam interagir com outras classes***, o que gera dependências e reduz a sua coesão).
- Criar classes que controlem as interações com outras classes de negócio.

E esta é apenas ***uma*** das possíveis implementações. Para cada item da lista, poderia haver repetição de nomes de campos, restrições de tabelas etc. — o que termina criando um bocado daquela indesejável redundância. Para atacar esses problemas, Django provê os chamados "modelos", que permitem definir as classes que representam entidades de negócio, como ***Produto***, ***NotaFiscal***, ***Cliente*** etc., em um mesmo local, sem criar redundância sobre redundância. Mais adiante mostrarei como fazê-lo ao construir uma aplicação.

1. ***Explícito é melhor que implícito***: Este conceito é emprestado do "Zen do Python". Uma das formas que Django implementa tal princípio é pela maneira como direciona as requisições dos clientes (browser ou outra aplicação) para os métodos que responderão a elas (só um "spoiler": esses métodos são definidos nas chamadas *Views*). Veja um exemplo na Listagem 14.1.

```
def visualizar_pedido(request, id_pedido):

pedido = Pedido.objects.get(id = id_pedido)

return render(request, 'pedidoo/pedido.html', {pedido: pedido})
```

LISTAGEM 14.1: Método que recupera um pedido

Esse é apenas um trecho de código, não um programa completo, e você ainda precisa aprender alguns conceitos antes de compreendê-lo totalmente, porém, perceba a ***legibilidade***: mesmo que não conheça o framework, é possível supor que esse código recebe um parâmetro id_pedido, recupera de algum lugar o pedido correspondente a esse id e devolve uma página HTML com o resultado. Nada fica oculto.

AVISO

Não se preocupe em entender agora o fragmento de código da Listagem 14.1 — ele foi colocado aqui apenas para ilustrar o princípio "explícito é melhor que implícito", sem nenhuma finalidade de ser completo neste momento.

Acoplamento fraco — Django divide sua aplicação em várias partes que operam em diversas camadas (interface com o usuário, regras de negócio, acesso a dados etc.), entretanto, elas foram pensadas para ***não misturar aspectos diferentes***: por exemplo, no código que acessa dados, existe apenas acesso a dados, nada de regras de negócio; na interface com o usuário, não há código de negócios etc.; observar essas práticas produz código menos "emaranhado" e mais fácil de manter.

Um pouco de arquitetura: o padrão MTV

O framework Django adota o padrão de projeto MTV (*Model, Template, View*), uma variação do conhecido padrão MVC (*Model, View, Controller*). Em termos simples, Django separa a camada de interface com o usuário de sua aplicação em três partes:

Modelo (Model)

Consiste na parte da aplicação que se comunica com o SGBD por meio do mapeamento objeto-relacional do Django. Durante muitos anos, um dos maiores problemas para quem criava aplicações para a web (e mesmo para outras plataformas) era como interagir de maneira transparente com o sistema que gerencia o banco de dados: escrever esse tipo de código partindo "do zero", apesar de possível, é extremamente complexo, propenso a erros e, ainda por cima, desnecessário — há várias ferramentas que simplificam essa tarefa fornecendo uma interface mais simples para as operações necessárias. Por exemplo: suponha que você crie uma tela com uma funcionalidade do tipo "Mestre-Detalhe", como um pedido e seus itens. Se fosse implementá-la sem um software de mapeamento OR, precisaria escrever código SQL para cada operação no banco de dados, código Python para receber as

Expandir tudo | Recolher tudo

Estudo 285
Empacotando uma Aplicação com ARDUINO IDE 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice A: Instalando as Ferramentas no Windows 425
Apêndice B: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

Consiste na parte da aplicação que se comunica com o SGBD por meio do mapeamento objeto-relacional do Django. Durante muitos anos, um dos maiores problemas para quem criava aplicações para a web (e mesmo para outras plataformas) era como interagir de maneira transparente com o sistema que gerencia o banco de dados: escrever esse tipo de código partindo "do zero", apesar de possível, é extremamente complexo, propenso a erros e, ainda por cima, desnecessário — há várias ferramentas que simplificam essa tarefa fornecendo uma interface mais simples para as operações necessárias. Por exemplo: suponha que você crie uma tela com uma funcionalidade do tipo "Mestre-Detalhe", como um pedido e seus itens. Se fosse implementá-la sem um software de mapeamento OR, precisaria escrever código SQL para cada operação no banco de dados, código Python para receber as informações das suas classes e mais código Python para converter uma representação em outra, afinal, espera-se que os programas Python usem um modelo de domínio orientado a objetos para representar seus dados, enquanto os SGBDs tipicamente usam um esquema relacional para armazená-los (daí o termo "objeto-relacional"). Você precisaria ainda controlar os dados para que, sempre que um registro mestre fosse visualizado, apagado ou editado, os registros filhos, na parte "detalhe" da tela, fossem modificados de acordo. Para resolver esses problemas, o Django possui o seu próprio mapeamento OR, chamado de Django ORM (Object-Relational Mapping). O framework pode, então, criar as tabelas no banco de dados para você, gerenciar operações DML (inclusão, alteração, exclusão e consulta de dados), controlar relacionamentos entre as tabelas que contêm informações etc. O Django suporta nativamente os SGBDs: PostgreSQL, MySQL, SQLite e Oracle. Além desses, existem também drivers fornecidos por terceiros, que permitem utilizar o SQL Server (Microsoft), DB2 (IBM), SQL Anywhere (SAP), Firebird e vários outros por meio de conexões ODBC.

DICA

Para os iniciantes ou mesmo usuários mais experientes que desejem realizar testes simples, recomendo usar o SQLite, porém ele não deve ser usado em produção. Quando você chegar ao ponto de lançar sua aplicação no mercado, duas boas escolhas gratuitas são o PostgreSQL e o MySQL. Grandes corporações usam muito Oracle e SQL Server. No entanto, mesmo um sistema gigantesco consegue rodar bem com o PostgreSQL ou o MySQL, se bem configurados.

Template

É a parte que o usuário visualiza, a ***camada de apresentação*** do sistema escrito com o Django. Os templates Django controlam a ***lógica de apresentação*** dos seus dados, ou seja, como eles serão visualizados pelo usuário, separando os dados da aplicação da maneira como são exibidos. Embora você possa criar seus templates com várias tecnologias diferentes, a maneira mais comum de implementar essa camada de software é por meio de páginas HTML decoradas com folhas de estilo em cascata (***CSS — Cascading Style Sheets***). Os templates isolam a lógica de negócios da apresentação, permitindo que, se necessário, um ***designer*** trabalhe na criação da identidade visual do sistema enquanto um programador implementa as funcionalidades separadamente. Além disso, não é permitido executar código Python em templates, o que evita brechas de segurança.

De fato, existe toda uma ***linguagem de templates*** embutida no Django, denominada ***DTL (Django Template Language)***, uma linguagem de script que usa tags para formatar o conteúdo a ser exibido. Por exemplo, imagine um trecho de página como o da Listagem 14.2:

```
<div>
<p> Usuário conectado:
<span class="username"> {{nome_usuario}} </span>
</p>
</div>
```

LISTAGEM 14.2: Trecho de template Django

Perceba que são usadas tags do HTML padrão (, e <p>), e a única novidade é uma variável, ***nome_usuario***, exibida entre {{ e }}. Essa é a maneira da linguagem de templates do Django exibir o conteúdo de uma informação, que é passada para o ***template*** pela ***view*** (não se preocupe em compreender os detalhes dessa operação agora: ela ficará mais clara quando eu criar o projeto-exemplo do livro). Esse trecho de código poderia estar presente em uma página que mostrasse o usuário atualmente logado em uma aplicação.

A DTL permite, dessa forma, separar os trabalhos do desenvolvedor, que cria os códigos internos da aplicação, e do designer, que se preocupa com a apresentação das informações: o primeiro não precisa se preocupar em como as informações serão fornecidas, apenas em quais variáveis irá expor à camada de apresentação; enquanto o segundo cuidará apenas do que sabe fazer bem, que é criar páginas organizadas e visualmente atraentes, sem se importar com o código que busca e atualiza informações.

Herança de Templates

Expandir tudo | Recolher tudo
Estilo 255
Empacotando uma Aplicação com AMQP E XP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

como as informações serão fornecidas, apenas em quais variáveis irá expor à camada de apresentação; enquanto o segundo cuidará apenas do que sabe fazer bem, que é criar páginas organizadas e visualmente atraentes, sem se importar com o código que busca e atualiza informações.

Herança de Templates

Lembra do princípio ***DRY (Don't Reppeat Yourself)***? Sua influência na DTL é percebida na chamada ***herança de templates***, recurso que permite "fatiar" o layout de uma página e reaproveitá-lo em vários lugares (ou, se preferir o termo técnico, permite ***herdar*** do layout padronizado). Por exemplo, a maioria das páginas de compras costuma ser organizada como uma variação do formato mostrado na Figura 14.1.

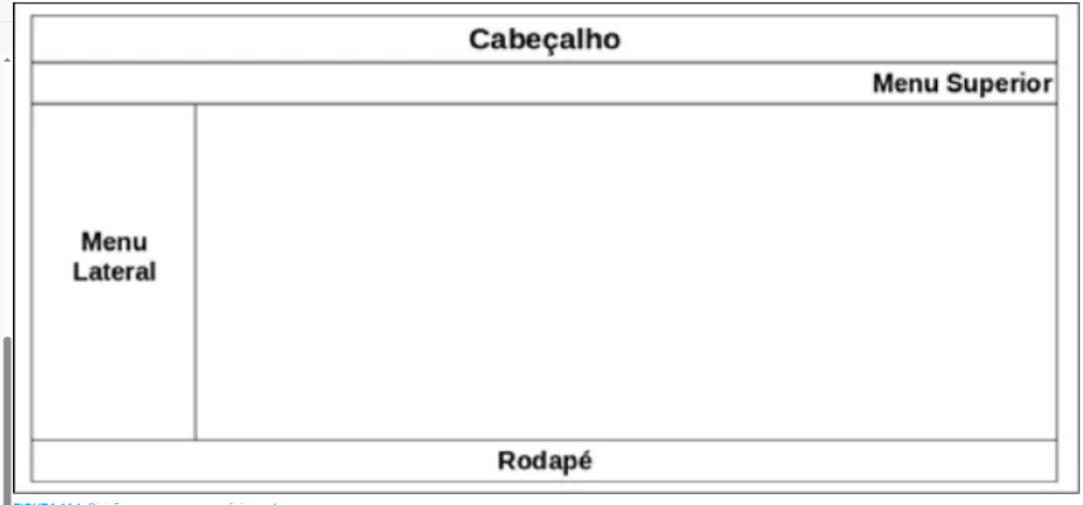


FIGURA 14.1: Divisões comuns em uma página web

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

Estilo 255
Empacotando uma Aplicação com AMQP e RP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice A: Instalação de Ferramentas no Windows 425
Apêndice B: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

FIGURA 14.1: Divisões comuns em uma página web

Se você alguma vez já teve que atualizar páginas de um site ***realmente grande***, sabe o pesadelo que isso pode se tornar se adotar a abordagem "copia, cola, altera". Isso simplesmente não funciona quando a quantidade de arquivos começa a aumentar muito: há código repetido demais. Se, por exemplo, a empresa muda de nome ou logomarca, seria necessário atualizar ***cada página;*** e é muito fácil que alguma informação seja esquecida ou alterada erroneamente no processo. Para prevenir isso, inicialmente você deve separar as partes fixas do seu layout daquelas que mudam em cada página, como na Figura 14.2.

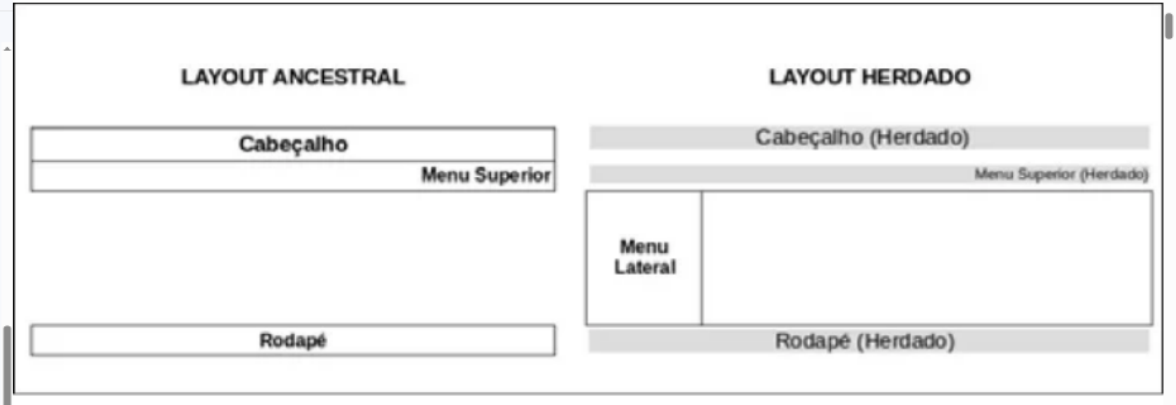


FIGURA 14.2: Layout "fatiado" mostrando parte ancestral e parte herdada

Você pode, ainda, herdar um layout de mais de um ancestral ao mesmo tempo (herança múltipla de templates): por exemplo, um template poderia definir o menu superior; outro, o rodapé; e assim sucessivamente.

AVISO

Não é permitida a execução de código Python ou atribuição de variáveis ***em um template***, para evitar brechas de segurança. Também não é permitido executar scripts de outras páginas para evitar ataques do tipo ***XSRF*** (***Cross-Site Request Forgery***) e ***XSS*** (***Cross-Site Scripting***). Esses tópicos estão fora do escopo deste livro, mas, se quiser saber mais a respeito, acesse (em inglês):

Expandir tudo | Recolher tudo

Estado 255
Empacotando uma Aplicação com AMQP e RP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

AVISO

Não é permitida a execução de código Python ou atribuição de variáveis ***em um template***, para evitar brechas de segurança. Também não é permitido executar scripts de outras páginas para evitar ataques do tipo ***XSRF*** (***Cross-Site Request Forgery***) e ***XSS*** (***Cross-Site Scripting***). Esses tópicos estão fora do escopo deste livro, mas, se quiser saber mais a respeito, acesse (em inglês):

- https://docs.djangoproject.com/en/2.2/ref/csrf/
- https://docs.djangoproject.com/en/2.2/topics/security/

Essas limitações aplicam-se apenas a código ***Python***. Não há limitações, por exemplo, à execução de código ***JavaScript*** na página.

View

A ***View*** corresponde, no Django, à camada que controla a interação entre os dados (***Modelo***) e a apresentação (***Template***). Ela embute as ***regras de negócio*** da sua aplicação.

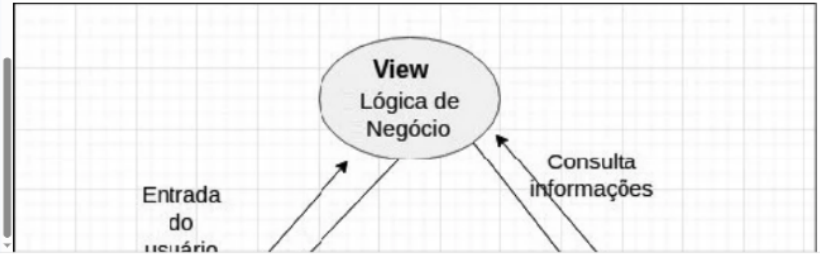
O nome escolhido para essa camada confunde muitos iniciantes, principalmente aqueles que vêm de outras linguagens orientadas a objetos, que usam o padrão ***MVC (Model-View-Controller)***, para os quais o componente que realiza essa tarefa é denominado de ***Controlador*** (***Controller***) e "***view***" é o nome dado ao que o Django denomina de ***template***. Se você for um deles, comece desde já a repetir mentalmente: "[...]no Django, uma ***View*** corresponde à camada de lógica da aplicação e ***NÃO*** à camada de apresentação."

DICA

a página web do framework, seus criadores colocaram uma nota sobre essa confusão entre MTV e MVC: "[...]no final do dia, claro, tudo se resume a conseguir que as coisas fiquem prontas. E, independentemente de como elas são nomeadas, Django consegue que elas fiquem prontas de um jeito que é, em sua maioria, lógico para nós." (Tradução livre de https://docs.djangoproject.com/en/2.2/faq/general/)

A Figura 14.3 mostra como essas três partes da aplicação interagem entre si.

Uma ***view*** normalmente devolve informações a um ***template*** na forma de uma variável ou um dicionário, para que seja montada uma página HTML de resposta a uma requisição. Entretanto, com o crescente uso de APIs REST,[4] de uns anos para cá, também se tornou comum que as visões devolvam dados em formato ***JSON*** (***JavaScript Object Notation***), muito usados com tais APIs.



Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

Estado 255
Empacotando uma Aplicação com ARDUINO IDE 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice A: Instalando as Ferramentas no Windows 425
Apêndice B: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

Uma *view* normalmente devolve informações a um *template* na forma de uma variável ou um dicionário, para que seja montada uma página HTML de resposta a uma requisição. Entretanto, com o crescente uso de APIs REST,[4] de uns anos para cá, também se tornou comum que as visões devolvam dados em formato *JSON* (*JavaScript Object Notation*), muito usados com tais APIs.

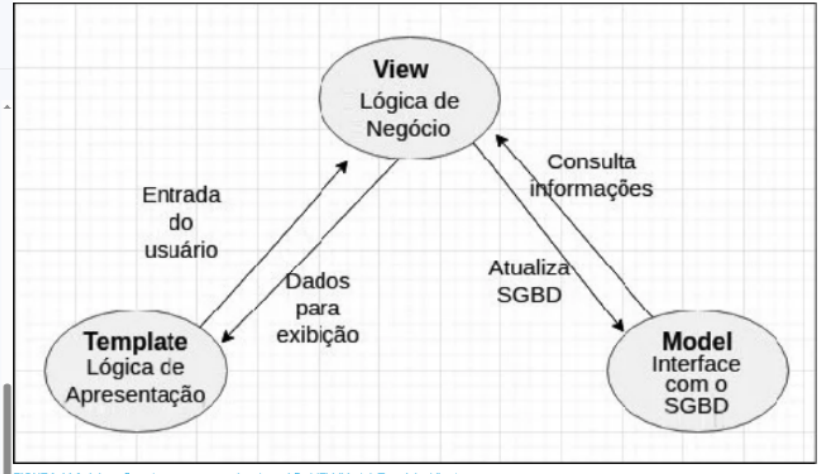


FIGURA 14.3: Interação entre os componentes do padrão MTV (Model, Template, View)

Uma analogia bastante utilizada para explicar o conceito da view em Django é pensar nela como uma espécie de "guarda de trânsito". A view controla quais requisições podem seguir de um lugar para outro e quais classes atenderão à solicitação. Ela "decide" quais dados serão mandados para um template, seja respondendo a uma ação tomada pelo usuário na camada de apresentação (*template*), seja respondendo a algum método de negócio interno à aplicação.

As views podem ser de dois tipos:

Expandir tudo | Recolher tudo

Estilo 253
Empurrando uma Aplicação com AMQP E JP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalando as Ferramentas no Windows 425
Apêndice 2: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

FIGURA 14.3: Interação entre os componentes do padrão MTV (Model, Template, View)

Uma analogia bastante utilizada para explicar o conceito da view em Django é pensar nela como uma espécie de "guarda de trânsito". A view controla quais requisições podem seguir de um lugar para outro e quais classes atenderão à solicitação. Ela "decide" quais dados serão mandados para um template, seja respondendo a uma ação tomada pelo usuário na camada de apresentação (***template***), seja respondendo a algum método de negócio interno à aplicação.

As views podem ser de dois tipos:

1. ***Views baseadas em funções*** (***function–based views***): Como o nome sugere, são representadas por funções escritas em Python. Adotam uma abordagem mais próxima da programação procedural. Há quatro ***views*** baseadas em funções predefinidas, usadas para exibir páginas de erro:
   - 400 (***Bad request***, requisição malformada)
   - 403 (***Forbidden***, endereço proibido)
   - 404 (***Page not found***, página não encontrada)
   - 500 (***Server error***, erro interno no servidor)

Para responder a esses tipos de erro usando ***function-based views***, sua aplicação deverá definir funções para cada um dos eventos citados que você deseje tratar.

2. ***Views baseadas em classes*** (***class–based views***): Introduzidas na versão 1.7 do Django, as ***class–based views*** são implementadas por meio de uma classe, ou seja, usam os recursos da ***orientação a objetos***. Há várias dessas views no Django, como, por exemplo:
   - ***FormView***: Permite exibir um formulário.
   - ***ListView***: Usada para exibir uma lista de objetos.
   - ***DetailView***: Serve para mostrar um só objeto.
   - etc.

Uma vantagem desse tipo de view é a possibilidade de usar ***herança***, assim você consegue reaproveitar comportamentos definidos em outras views.

AVISO

Você poderia até mesmo ignorar completamente a arquitetura recomendada e criar uma aplicação com todos os componentes em um único arquivo,[2] porém, quando seu projeto começar a crescer — pois a única coisa constante em desenvolvimento de software é a mudança — você terá em mãos um "pesadelo de manutenção".

Configurações de URLs

Finalmente, a "cola" que une as três partes apresentadas anteriormente são as configurações de URL.[3] Elas especificam a origem e o destino das requisições que serão processadas pelas views da aplicação.

Toda página web tem sua URL, elas foram criadas para que você não precise decorar endereços numéricos, afinal, é bem mais fácil lembrar de www.qualquercoisa.com que de 192.168.1.250, não é verdade?[4]

Esses endereços são especificados, ***por padrão***, por meio de um mapeamento presente em um arquivo denominado urls.py.

AVISO

Expandir tudo | Recolher tudo

Resumo 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalando as Ferramentas no Windows 425
Apêndice 2: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

Finalmente, a "cola" que une as três partes apresentadas anteriormente são as configurações de URL's. Elas especificam a origem e o destino das requisições que serão processadas pelas views da aplicação.

Toda página web tem sua URL, elas foram criadas para que você não precise decorar endereços numéricos, afinal, é bem mais fácil lembrar de www.qualquercoisa.com que de 192.168.1.250, não é verdade?[4]

Esses endereços são especificados, ***por padrão***, por meio de um mapeamento presente em um arquivo denominado urls.py.

AVISO

Escrevi que os endereços se localizam em urls.py "por padrão" porque, de fato, você poderia defini-los em outro local. Uma característica muito forte no Django Framework é sua capacidade de ***personalização*** — você pode seguir o padrão (e, acredite: há muitas vantagens em fazê-lo, por isso, recomendo isso na maioria dos casos), mas, se houver alguma boa razão para não fazê-lo, o ***framework*** não ficará no seu caminho.

Criando um novo projeto com o Django

OK. Chegou a hora de pôr a "mão na massa": vou mostrar como criar o seu primeiro projeto Django. Se ainda não instalou o Django e o Python, dê um "pulinho" no apêndice 1 ou 2, dependendo de qual sistema operacional você utiliza, instale-os e retorne a esta página.

AVISO

Não salte sem paraquedas! Em outras palavras, não comece um projeto sem configurar um ambiente virtual para trabalhar. Embora alguns programadores sintam-se mais confortáveis sem eles, os ambientes virtuais fornecem a garantia mínima de que não haverá interações com *softwares*/bibliotecas instaladas na máquina de desenvolvimento que possam interferir no seu trabalho. Nos apêndices, você encontrará informações sobre como configurar um ambiente virtual para trabalhar.

Para criar um projeto, antes de mais nada, ***inicie o seu ambiente virtual***, conforme ensinado nos apêndices. Entre no seu prompt de comando, acesse a pasta em que pretende salvar seus projetos Django e digite:

```
django-admin startproject nome_de_projeto
```

No meu caso, chamei o projeto de "lojavirtual", logo, digites:

```
django-admin startproject lojavirtual
```

O Django vem com um servidor web interno apenas para testes. Não é poderoso ou seguro o suficiente para funcionar em um ambiente de produção, mas, para aprendizado, serve bem aos nossos objetivos. Se você não fizer mais nada agora e rodá-lo, já poderá ver a tela padrão do Django. Experimente digitar:

```
cd lojavirtual

./manage.py runserver
```

Serão exibidas algumas mensagens parecidas com:

```
Watching for file changes with StatReloader

Performing system checks...

System check identified no issues (0 silenced).

You have 17 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.

Run 'python manage.py migrate' to apply them.

July 01, 2019 - 20:21:29

Django version 2.2.1, using settings 'lojavirtual.settings'

Starting development server at http://127.0.0.1:8000/

Quit the server with CONTROL-C.
```

Expandir tudo | Recolher tudo

Estudo 255
Empacotando uma Aplicação com PIPENV e VIP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

```
July 01, 2019 - 20:21:13
Django version 2.2.1, using settings 'lojavirtual.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

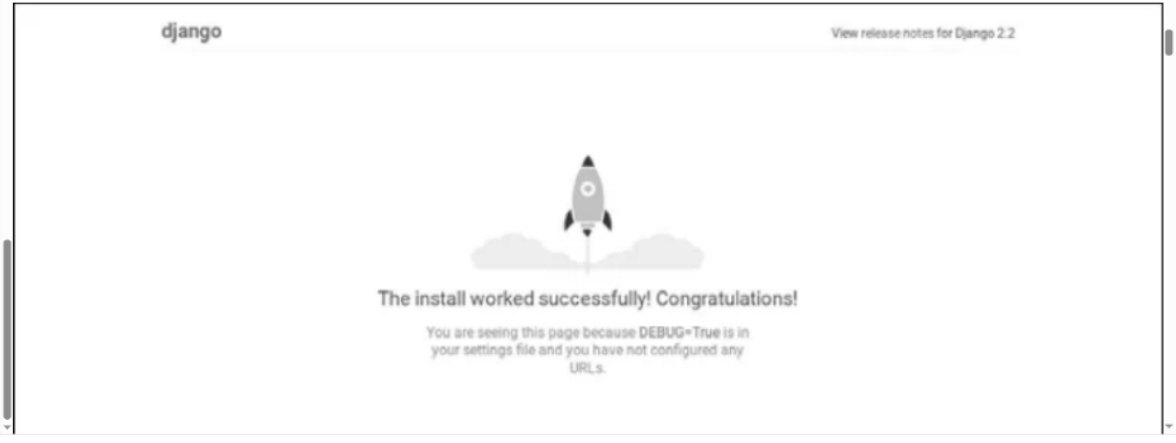
As linhas em destaque informam que ainda não foram criadas as tabelas iniciais, no banco de dados da aplicação, que serão usadas pelo Django. Interrompa o servidor digitando ctrl+C.
Para criar essas tabelas, digite:

```
./manage.py migrate
```

Aparecerão várias mensagens de confirmação durante a criação das estruturas no banco de dados. Agora, repita o comando:

```
./manage.py runserver
```

Nesse momento, se você acessar no browser o endereço mostrado (http://127.0.0.1:8000/), verá a tela inicial de uma aplicação Django, como na Figura 14.4.



Expandir tudo | Recolher tudo

Estudo 255

Empacotando uma Aplicação com PIPEX E VIP 263

O Django Framework 279

Modelos 309

Formulários e Templates 337

Criando Telas de Cadastro com o Django Admin 371

Construindo a Aplicação de Loja Virtual com o Django 377

Apêndice 1: Instalação de Ferramentas no Windows 425

Apêndice 2: Instalação de Ferramentas no Linux 430

Referências Bibliográficas 433

FIGURA 14.4: Tela inicial de uma aplicação Django

Pode não parecer muita coisa, mas pense a respeito: sem nenhum esforço, você exibiu uma página de boas-vindas que contém links funcionais para outras páginas.

Antes de alterar o projeto para nossos propósitos, ainda na pasta ***lojavirtual***, dê uma olhada nos arquivos e pastas existentes:

```
\lojavirtual (pasta raiz do projeto)
db.sqlite3
manage.py
\lojavirtual (configurações do projeto)
__init__.py
settings.py
urls.py
wsgi.py
```

A pasta raiz, lojavirtual, serve apenas para agrupar os arquivos e pastas do projeto. Ela é conhecida pelo nome de ***BASE_DIR***. Você pode renomeá-la para qualquer coisa que desejar, sem medo de "quebrar" algo — na verdade, recomendo que o faça — essas duas pastas com o mesmo nome costumam criar mais confusão que ajudar. No meu caso, eu a renomeei para "***prj_lojavirtual***", deixando a estrutura do projeto assim:

```
\prj_lojavirtual (pasta raiz do projeto)
db.sqlite3
manage.py
```

Expandir tudo | Recolher tudo

Estado 263

Formatando uma Aplicação com AMDG E JQ 279

O Django Framework 309

Modelos 337

Formulários e Templates 371

Criando Telas de Cadastro com o Django Admin 377

Concluindo a Aplicação de Loja Virtual com o Django 425

Apêndice 1: Instalando as Ferramentas no Windows 430

Apêndice 2: Instalando as Ferramentas no Linux 433

Referências Bibliográficas

urls.py

wsgi.py

A pasta raiz, lojavirtual, serve apenas para agrupar os arquivos e pastas do projeto. Ela é conhecida pelo nome de ***BASE_DIR***. Você pode renomeá-la para qualquer coisa que desejar, sem medo de "quebrar" algo — na verdade, recomendo que o faça — essas duas pastas com o mesmo nome costumam criar mais confusão que ajudar. No meu caso, eu a renomeei para "***prj_lojavirtual***", deixando a estrutura do projeto assim:

\prj_lojavirtual (pasta raiz do projeto)

db.sqlite3

manage.py

\lojavirtual (configurações do projeto)

__init__.py

settings.py

urls.py

wsgi.py

Quanto ao conteúdo do projeto:

- O arquivo ***db.sqlite3*** é um gerenciador de bancos de dados em memória (SQLite) usado para testes simples e apenas em ambiente de ***desenvolvimento***.
- ***manage.py*** é um script que serve para executar tarefas administrativas da sua aplicação por linha de comandos (uma dessas tarefas, ***runserver***, permite executar o servidor web interno do Django, como visto no início desta seção).

A pasta "***lojavirtual***", mais interna, é citada em muitos livros como ***PROJECT_DIR***. Ela contém configurações que afetam o projeto como um todo. Nela, você encontrará os arquivos:

- ***__init__.py***: Usado para marcar essa pasta como um pacote Python. Se você não se recorda do conceito, releia a seção "Agrupando módulos em pacotes", no Capítulo 9.
- ***settings.py***: Este arquivo contém informações de ***configuração*** sobre seu projeto. Mais detalhes a seu respeito adiante, na seção "Ajustando o settings.py".
- ***urls.py***: Contém mapeamentos de URLs internas ao projeto. Qualquer URL que precise ser acessada dentro do projeto deve ser mapeada nele. Logo que você cria um projeto, há apenas um mapeamento para a página de administração do site, criada automaticamente pelo Django.
- ***wsgi.py***: Permite que seu projeto seja implantado em servidores compatíveis com o padrão WSGI, usado em servidores web otimizados para Python.

AVISO

Cuidado para não se confundir! A pasta que você pode renomear é a BASE_DIR. ***JAMAIS*** renomeie a pasta mais interna, conhecida como ***PROJECT_DIR***. Se você o fizer, provocará erros em diversos arquivos como settings.py e wsgi.py.

DICA

Todos os arquivos de configuração do Django são, na realidade, ***programas escritos em Python***, então você perceberá a extensão .py e a sintaxe familiar de listas e dicionários para especificar os parâmetros necessários.

Por isso, ao planejar este livro, decidi lançar formar sólidos conceitos do Python na primeira parte — você poderia iniciar no Django com um conhecimento apenas superficial da linguagem, mas, se investir em

Expandir tudo | Recolher tudo

Resumo 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação da Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

DICA

Todos os arquivos de configuração do Django são, na realidade, ***programas escritos em Python***, então você perceberá a extensão .py e a sintaxe familiar de listas e dicionários para especificar os parâmetros necessários.

Por isso, ao planejar este livro, decidi tentar formar sólidos conceitos de Python na primeira parte — você poderia iniciar no Django com um conhecimento apenas superficial da linguagem, mas, se investiu em conhecê-la um pouco melhor, poderá aproveitar muito mais os recursos do framework.

Adicionando uma aplicação ao projeto

Os projetos Django são subdivididos em aplicações. Cada aplicação é uma seção autocontida do seu projeto. Pense em uma aplicação como uma seção de um site: se, por exemplo, você criar um site de notícias, este será o seu ***projeto***. Cada seção do site (Cinema, Esportes, Política etc.) poderia ser criada como uma ***aplicação***. Dessa maneira, tais aplicações poderiam ser reaproveitadas em outros projetos (por exemplo, se você precisar criar uma versão diferente da página para outro cliente, poderá reaproveitar as aplicações que já desenvolveu para o projeto anterior).

DICA

Como já citei, Django surgiu como um gerenciador de conteúdo para portais de notícias e, para esse tipo de site, faz muito sentido a estruturação em projetos e aplicações: o ***projeto*** corresponde ao portal inteiro e seções como "Diversão", "Esportes", "Política" e "Exterior" podem ser criadas como aplicações separadas.

Isso não significa, entretanto, que tal organização seja ruim para outros tipos de sistemas — muito pelo contrário. Você descobrirá, com o tempo, que essa organização facilita muito a manutenção do seu código.

Comece simples: crie seus projetos sem se preocupar com o que será aplicação e o que não será. À medida que o projeto for avançando, você perceberá se existem partes que são suficientemente coesas para merecerem sua própria aplicação (claro, às vezes a tarefa é tão grande que, já na fase de projeto, é interessante definir algumas das ***apps*** que serão criadas). Se precisar criar uma app, use o *manage.py*:

manage.py startapp ***nome_da_app***

Crie uma aplicação chamada ***main***, que será a primeira do projeto Loja Virtual: para tal, vá ao prompt de comandos/terminal, dentro da pasta raiz do seu projeto (***prj_lojavirtual***) e digite:

```
manage.py startapp main
```

Será gerada uma nova subpasta dentro da pasta principal do projeto, com o nome de ***main***. Perceba que a pasta main ficará dentro de ***prj_lojavirtual***, no mesmo nível da pasta de configurações, ***lojavirtual***. Como o arquivo do projeto originalmente também se chamava ***lojavirtual***, os novatos costumam se perder na nomenclatura (esse foi o motivo de eu ter orientado você a renomear a pasta raiz do projeto logo no início).

AVISO

Se, por engano, você mover a pasta da nova aplicação, ***main***, para outro local, por exemplo, para dentro de ***prj_lojavirtual\lojavirtual***, receberá vários erros em tempo de execução ao tentar rodar o código que usa essa aplicação; e encontrar a origem dos erros pode ser uma verdadeira "dor de cabeça" se tiver criado uma aplicação com um nome parecido com o do projeto.

Estrutura de uma aplicação Django

Dê uma olhada no conteúdo da pasta main:

```
\main
\migrations
admin.py
apps.py
models.py
tests.py
```

Expandir tudo | Recolher tudo

263
279
309
337
371
377
425
430
433

Dê uma olhada no conteúdo da pasta main:

```
\main
\migrations
admin.py
apps.py
models.py
tests.py
views.py
__init__.py
```

A pasta ***migrations*** é usada para armazenar scripts de migração de bancos de dados. Esses scripts são criados e executados automaticamente pelo framework, mas também podem ser personalizados pelo programador.

Os arquivos admin.py e models.py são usados para descrever suas classes de domínio que o Django chama de ***modelos*** para serem usadas na aplicação Admin, que acompanham o framework. Isso permite criar, muito rapidamente, telas com funcionalidades ***CRUD***, usadas para manter cadastros simplificados.

O arquivo tests.py serve para definir os testes que farão parte da aplicação.

O arquivo views.py descreve as views da aplicação.

Começarei as configurações do projeto pelo arquivo apps.py. Se você examinar seu código, verá que ele se resume ao mostrado na Listagem 14.3:

```
from django.apps import MainConfig

class MainConfig(AppConfig):

    name = 'main'
```

LISTAGEM 14.3: Conteúdo do arquivo apps.py, na aplicação main

O que nos interessa, nesse fragmento, é a classe gerada, ***MainConfig***, que herda de ***AppConfig***. Para que a aplicação possa ser acessada no projeto do qual faz parte, ela deve ser adicionada à lista apontada pela constante ***INSTALLED_APPS***, no arquivo ***settings.py*** do projeto atual.

Ajustando o settings.py

No arquivo settings.py você encontrará "toneladas" de ajustes possíveis, o que, por vezes, causa confusão entre iniciantes: conexões com bancos de dados, configurações de ***caching***, internacionalização e carregamento de recursos estáticos, como imagens e folhas de estilo CSS.

Há várias ***constantes*** no arquivo, todas digitadas em ***MAIÚSCULAS*** — são consideradas configurações e são acessíveis em qualquer parte do projeto. Para usá-las em suas aplicações, importe a classe ***django.conf.settings***. Por exemplo, logo no início do settings.py, é definida a constante ***BASE_DIR***, que aponta para a pasta raiz do projeto (no caso atual, ***\prj_lojavirtual***).

Existe, ainda, um módulo global_settings.py, localizado na pasta conf da sua instalação do Django (por exemplo, se você instalou o framework em c:\django, o arquivo estará em c:\django\conf), que contém configurações padrão para todas as aplicações. Se um parâmetro não estiver especificado no settings.py da aplicação, será usado o valor encontrado em global_settings.py.

Discutirei, nas seções seguintes, as configurações mais utilizadas em aplicações reais. Uma referência completa pode ser obtida na documentação do Django (https://docs.djangoproject.com/en/2.2/topics/settings/).

Ligando/desligando o modo de debug do Django

Uma constante em particular no settings.py merece sua atenção logo de início: ***DEBUG***. Quando ajustada para True (valor padrão), sempre que ocorrer um erro em seu projeto, várias informações úteis para depurar o problema serão exibidas no browser. A Figura 14.5 mostra um exemplo.

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

263
279
309
337
371
377
425
430
433

Ligando/desligando o modo de debug do Django

Uma constante em particular no settings.py merece sua atenção logo de início: ***DEBUG***. Quando ajustada para True (valor padrão), sempre que ocorrer um erro em seu projeto, várias informações úteis para depurar o problema serão exibidas no browser. A Figura 14.5 mostra um exemplo.

FIGURA 14.5: Tela de erro mostrando informações de depuração

AVISO

Ao lançar seu código para produção, ***SEMPRE*** deixe *DEBUG = False* no settings.py. Você não quer que hackers recebam "de presente" informações sobre a estrutura interna de sua aplicação, não é verdade?

Configurando o idioma

Outro ajuste recomendável no settings.py é alterar a constante que define o ***idioma*** da aplicação:

```
LANGUAGE_CODE = 'pt-br'
```

Com esse ajuste, as telas de administração do Django, que usarei mais adiante para criar formulários de entrada de dados, serão traduzidas para o nosso português.

Ajustando o seu fuso horário

Logo que cria o projeto, ele é ajustado para o fuso horário "UTC", o fuso horário de referência para o mundo inteiro. Se está no Brasil, como eu, neste momento, altere a constante TIME_ZONE para:

```
TIME_ZONE = 'America/Sao_Paulo'
```

Que, apesar do título, é o horário de Brasília (cuidado para não digitar os valores com acentos).

Aplicações instaladas no seu projeto

Expandir tudo | Recolher tudo

Estudo 256
Empacotando uma Aplicação com AMQP e RP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalando as Ferramentas no Windows 425
Apêndice 2: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

Ajustando o seu fuso horário

Logo que cria o projeto, ele é ajustado para o fuso horário "UTC", o fuso horário de referência para o mundo inteiro. Se está no Brasil, como eu, neste momento, altere a constante TIME_ZONE para:

```
TIME_ZONE = 'America/Sao_Paulo'
```

Que, apesar do título, é o horário de Brasília (cuidado para não digitar os valores com acentos).

Aplicações instaladas no seu projeto

Como já citado, um projeto Django é composto por ***aplicações***. No momento de sua criação, o projeto já conta com as aplicações mostradas na Listagem 14.4.

```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
]
```

LISTAGEM 14.4: Aplicações declaradas por padrão no settings.py

Nas versões anteriores do Django, para ativar uma nova aplicação em seu projeto, bastava adicionar uma linha com o seu nome à lista ***INSTALLED_APPS***. Para modularizar mais os projetos, as versões recentes exigem que você especifique uma classe que herde de AppConfig. Ao usar o comando ***startapp*** do ***manage.py***, ele já cria essa subclasse para você no arquivo ***apps.py*** da sua aplicação. Entretanto, é ***sua responsabilidade*** incluir uma referência à aplicação no arquivo settings.py de sua pasta de configurações.

Como o arquivo todo é um programa em Python, e a constante ***INSTALLED_APPS*** aponta para uma lista de strings, simplesmente adicione a classe ***MainConfig*** à lista mostrada na Listagem 14.4, deixando-a como na Listagem 14.5.

```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'main.apps.MainConfig',
]
```

LISTAGEM 14.5: Trecho de settings.py mostrando a aplicação main

Especificando o caminho para arquivos estáticos do seu projeto

Expandir tudo | Recolher tudo

Estado 255
Preparando uma Aplicação com AMDG e TP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice A: Instalando as Ferramentas no Windows 425
Apêndice B: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

```
)
```

LISTAGEM 14.5: Trecho de settings.py mostrando a aplicação main

Especificando o caminho para arquivos estáticos do seu projeto

Qualquer aplicação web possui, entre seus recursos, arquivos estáticos (imagens, folhas de estilo, scripts etc.).

No final do arquivo settings.py, você encontrará este trecho:

```
# Static files (CSS, JavaScript, Images)

# https://docs.djangoproject.com/en/2.2/howto/static-files/

STATIC_URL = '/static/'
```

Essa constante diz ao framework para concatenar a URL do servidor com o valor fornecido sempre que o endereço de um arquivo estático for solicitado. No projeto atual, o resultado seria http://127.0.0.1:8000/static/.

Você pode armazenar seus arquivos estáticos em praticamente qualquer local do servidor, porém, por uma questão de simplicidade, eu os guardarei em uma pasta chamada ***static*** dentro do BASE_DIR da aplicação (ou seja, o caminho da pasta será ***/prj_lojavirtual/static***). Para informar isso ao Django, adicione ao seu settings.py:

```
STATICFILES_DIRS = (

os.path.join(BASE_DIR, 'static'),

)
```

Lembre-se de que o settings.py é, em última análise, um ***programa em Python***, e como tal, nele valem as mesmas regras de sintaxe que você usou ao longo do livro até o Capítulo 11. Assim, ***STATICFILES_DIRS*** é uma ***tupla*** e, portanto, pode conter vários elementos, o que implica que podem ser acrescentadas múltiplas localizações de arquivos nessa constante.

A constante ***STATIC_ROOT***, por sua vez, especifica o local em que devem ser armazenados os arquivos estáticos do projeto no momento de distribuir sua aplicação para um servidor web como ***GUnicorn*** ou ***Apache***. Quando em produção, esses arquivos serão gerenciados pelo servidor HTTP. Costumo especificar uma pasta separada, na raiz do ***projeto***, para esses arquivos. Para isso, é só adicionar ao settings.py:

```
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

Em que staticfiles é o nome da pasta na qual esses arquivos ficarão localizados.

DICA

Você não precisa criar, antecipadamente, a pasta ***staticfiles*** citada; ela será gerada automaticamente no momento de instalar sua aplicação em um servidor web. Para que o Django crie essa pasta e coloque todos os arquivos estáticos do projeto nela, digite:

```
./manage.py collectstatic
```

(Basta fazer isso na hora de instalar os arquivos finais no servidor de destino).

Especificando o caminho para arquivos estáticos fornecidos pelos usuários

O Django diferencia os arquivos estáticos fornecidos pelos usuários por meio de uploads, daqueles que foram criados junto com a aplicação. Em ambiente de ***desenvolvimento***, eles serão buscados na pasta apontada por ***MEDIA_ROOT***. Você também pode definir uma constante, ***MEDIA_URL***, que será usada para preceder os nomes desses arquivos, que ficarão disponíveis para download no endereço formado por ***MEDIA_URL***, seguida pela URL do próprio arquivo. Por exemplo, se sua pasta para arquivos estáticos chamar-se ***upload*** e estiver localizada no ***BASE_DIR***, você poderia ter a seguinte configuração:

```
MEDIA_ROOT = os.path.join(BASE_DIR, 'upload')

MEDIA_URL = '/upload/'
```

Expandir tudo | Recolher tudo

Resumo 263
Preparando uma Aplicação com AMDG e JP 279
O Django Framework 309
Modelos 337
Formulários e Templates 371
Criando Telas de Cadastro com o Django Admin 377
Construindo a Aplicação de Loja Virtual com o Django 425
Apêndice 1: Instalação de Ferramentas no Windows 430
Apêndice 2: Instalação de Ferramentas no Linux 433
Referências Bibliográficas

URL do próprio arquivo. Por exemplo, se sua pasta para arquivos estáticos chamar-se ***upload*** e estiver localizada no ***BASE_DIR***, você poderia ter a seguinte configuração:

```
MEDIA_ROOT = os.path.join(BASE_DIR, 'upload')

MEDIA_URL = '/upload/'
```

A constante MIDDLEWARE

Middlewares, para o Django, são componentes que adicionam funcionalidades ao ciclo de requisição — resposta do protocolo HTTP, ou seja, sempre que você digitar um endereço de um recurso gerenciado pelo Django em seu browser, se houver um middleware configurado para interceptar essa solicitação, ele será usado. Uma das utilizações mais comuns desse recurso é para autenticar (conferir login e senha, por exemplo) e autorizar (verificar se o usuário autenticado tem permissão para acessar determinada página do projeto) usuários, o que é feito pelos middlewares SessionMiddleware e AuthenticationMiddleware.

AVISO

Alguns componentes de terceiros, para serem instalados, requerem que linhas sejam adicionadas à lista apontada pela constante *MIDDLEWARE*. ***Cuidado ao fazê-lo!*** A *ordem* da lista afeta o resultado. Só adicione linhas ao final dela.

Templates

O Django suporta diversos mecanismos de template para gerar suas páginas. Em geral, fico com as configurações padrão. Se você utilizar um mecanismo alternativo, verifique como configurá-lo no seu manual. Há, porém, um ajuste que costumo fazer: alterar a pasta em que o Django procurará pelos seus templates.

É necessário, antes de alterar essa pasta, conferir se ***sua aplicação está presente na lista apontada pela constante INSTALLED_APPS***. Em seguida, acrescente a localização da pasta desejada ao arquivo de configurações. Há várias possibilidades de organização dos seus templates. Eu gosto de colocá-los em uma subpasta da ***raiz do projeto***, denominada ***templates***. Para isso, ajuste seu settings.py como na Listagem 14.6:

```
# Começo do arquivo omitido
TEMPLATES = [
{
'BACKEND': 'django.template.backends.django.
DjangoTemplates',
'DIRS': ['templates'],
'APP_DIRS': True,
'OPTIONS': {
'context_processors': [
'django.template.context_processors.debug',
'django.template.context_processors.request',
'django.contrib.auth.context_processors.auth',
'django.contrib.messages.context_processors.
messages',
],
},
```

Expandir tudo | Recolher tudo

Estilo 255

Empacotando uma Aplicação com MVC E TP 263

O Django Framework 279

Modelos 309

Formulários e Templates 337

Criando Telas de Cadastro com o Django Admin 371

Construindo a Aplicação de Loja Virtual com o Django 377

Apêndice 1: Instalando as Ferramentas no Windows 425

Apêndice 2: Instalando as Ferramentas no Linux 430

Referências Bibliográficas 433

```
    'django.contrib.messages.context_processors',
    'messages',
    ],
    },
    },
    ]
    TEMPLATE_DIRS = (
    os.path.join(BASE_DIR, 'templates'),
    )
    # Continuação...
```

LISTAGEM 14.6: Trecho de settings.py, mostrando a configuração dos templates

Para projetos pequenos, como é o caso do exemplo deste livro, é interessante colocar todos os arquivos de templates agrupados sob o ***projeto***. Porém, dependendo do porte e das decisões de projeto adotadas para sua aplicação, você pode querer usar uma configuração diferente para suas pastas.

DICA

O comportamento padrão do framework ao procurar por templates é:

1. O primeiro lugar em que se buscará os templates será a pasta apontada pela constante ***TEMPLATE_DIRS***, dentro do ***BASE_DIR***.
2. Se a pasta do item 1 não existir *E* a chave ***TEMPLATES['APP_DIRS']*** contiver o valor ***True***, ele buscará por uma pasta com o nome de ***templates*** nas pastas das ***aplicações registradas, na ordem em que foram declaradas na constante INSTALLED_APPS, no arquivo settings.py***. No caso do projeto prj_lojavirtual, deixei a chave ***TEMPLATES['APP_DIRS']*** com o valor ***True*** porque usarei a aplicação ***Admin*** em outras partes do projeto e, sem essa configuração, ela não consegue encontrar seus templates.
3. Se a pasta do item 1 não existir *E* a chave ***TEMPLATES['APP_DIRS'] = False***, ao receber uma requisição para exibir um template, o Django lançará uma exceção ***TemplateDoesNotExist***.

Configurando o log de mensagens

Algo de que você ***não deve esquecer*** é de configurar o registro (***logging***)[5] de mensagens da sua aplicação. Se acontecer alguma falha em tempo de execução, o log será um grande aliado na descoberta da causa e correção. Um exemplo de configuração de logging é mostrado na Listagem 14.7.

```
LOGGING = {
'version': 1,
'disable_existing_loggers': False,
'formatters': {
'simple': {
'format': 'Mensagem: %(levelname)s %(message)s'
},
},
```

Expandir tudo | Recolher tudo

Estudo 255

Preparando uma Aplicação com AMQP E JP 263

O Django Framework 279

Modelos 309

Formulários e Templates 337

Criando Telas de Cadastro com o Django Admin 371

Construindo a Aplicação da Loja Virtual com o Django 377

Apêndice 1: Instalando as Ferramentas no Windows 425

Apêndice 2: Instalando as Ferramentas no Linux 430

Referências Bibliográficas 433

```
'version': 1,
'disable_existing_loggers': False,
'formatters': {
'simple': {
'format': 'Mensagem: %(levelname)s %(message)s'
},
},
'handlers': {
'console': {
'level': 'DEBUG',
'class': 'logging.StreamHandler',
'formatter': 'simple'
},
},
'loggers': {
'lojavirtual': {
'handlers': ['console'],
'level': 'DEBUG',
'propagate': True,
},
'main': {
'handlers': ['console'],
'level': 'DEBUG',
'propagate': True,
},
},
}
```

LISTAGEM 14.7: Configuração de logging em settings.py

A parte relevante dessa listagem são os dicionários contidos na chave "***loggers***": eu os chamei com o nome das aplicações criadas no projeto: ***lojavirtual*** e ***main***. A chave "***level***" foi configurada para ***DEBUG*** informando ao framework que só queremos que essas mensagens sejam enviadas ao console quando o servidor for iniciado em modo de ***debug***. Finalmente, "***propagate***" foi estabelecido como True para que as exceções que porventura ocorram

Expandir tudo | Recolher tudo

Estado 263
Preparando uma Aplicação com AMDG & JP 279
O Django Framework 309
Modelos 337
Formulários e Templates 371
Criando Telas de Cadastro com o Django Admin 377
Contribuindo a Aplicação de Loja Virtual com o Django 425
Apêndice 1: Instalação de Ferramentas no Windows 430
Apêndice 2: Instalação de Ferramentas no Linux 433
Referências Bibliográficas

```
'propagate': True,
},
},
}
```

LISTAGEM 14.7: Configuração de logging em settings.py

A parte relevante dessa listagem são os dicionários contidos na chave "***loggers***": eu os chamei com o nome das aplicações criadas no projeto: ***lojavirtual*** e ***main***. A chave "***level***" foi configurada para ***DEBUG*** informando ao framework que só queremos que essas mensagens sejam enviadas ao console quando o servidor for iniciado em modo de ***debug***. Finalmente, "***propagate***" foi estabelecido como True para que as exceções que porventura ocorram não sejam silenciadas.

Configurando um banco de dados

A grande maioria dos sistemas que serão desenvolvidos utilizará algum tipo de banco de dados para armazenar e recuperar informações. Alguns tipos de aplicações podem até prescindir de um SGBD, porém, costumam ser aplicações muito simples, sem grande apelo comercial.

O Django vem pré-configurado para utilizar o SQLite, como já citado, mas esse SGBD não é robusto o suficiente para suportar grandes aplicações com intenso tráfego de dados e muitos usuários. Felizmente, o framework também embute suporte a diversos SGBDs muito mais poderosos, tais como PostgreSQL, Oracle, MySQL, e ainda vários outros por meio de conexões ODBC.[6] Se precisar conectar-se a um SGBD diferente dos citados, consulte a documentação do Django no tópico https://docs.djangoproject.com/en/2.2/ref/databases/.

Para configurar o seu projeto para usar um SGBD diferente, edite o arquivo *settings.py* e procure a variável DATABASES. Ela contém um dicionário que especifica os parâmetros de acesso ao gerenciador de banco de dados escolhido. A Listagem 14.8 mostra esse trecho do arquivo.

```
DATABASES = {
'default': {
'ENGINE': 'django.db.backends.sqlite3',
'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
}
}
```

LISTAGEM 14.8. Configuração de banco de dados em settings.py

O dicionário tem uma chave, 'default', que, como o nome indica, aponta para a configuração padrão de banco de dados. Se nada diferente for especificado, qualquer operação de manipulação de dados será executada com esse SGBD. Tal chave aponta para um outro dicionário,[7] que define os parâmetros necessários ao driver de acesso ao SGBD escolhido.

No caso da configuração ***default***, os parâmetros exigidos são:

- ***engine***: Estabelece qual tipo de banco de dados (MySQL, Oracle, SQLite etc.) será acessado. A Tabela 14.1 mostra os possíveis valores para essa configuração. (Se você usa um SGBD não listado na tabela, consulte a documentação do Django no tópico https://docs.djangoproject.com/en/2.2/ref/databases/.)
- ***name***: Define qual instância do banco de dados será acessada. Pense nesse parâmetro como o nome do banco que sua aplicação usará (por exemplo, se você está construindo um sistema de vendas, a instância poderia se chamar "db_vendas", ou algo parecido). Para o SQLite, ***sempre*** passe o caminho completo do arquivo do banco de dados, usando o caractere de barra (/) para especificar pastas, mesmo no Windows (por exemplo: /apps/consulta/db.sqlite3).

Expandir tudo | Recolher tudo

Resumo 255
Empacotando uma Aplicação com AMQP e RP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação do Site Loja Virtual com o Django 377
Apêndice A: Instalando as Ferramentas no Windows 425
Apêndice B: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

```
}
}
```

LISTAGEM 14.8: Configuração de banco de dados em settings.py

O dicionário tem uma chave, 'default', que, como o nome indica, aponta para a configuração padrão de banco de dados. Se nada diferente for especificado, qualquer operação de manipulação de dados será executada com esse SGBD. Tal chave aponta para um outro dicionário,[2] que define os parâmetros necessários ao driver de acesso ao SGBD escolhido.

No caso da configuração ***default***, os parâmetros exigidos são:

- ***engine***: Estabelece qual tipo de banco de dados (MySQL, Oracle, SQLite etc.) será acessado. A Tabela 14.1 mostra os possíveis valores para essa configuração. (Se você usa um SGBD não listado na tabela, consulte a documentação do Django no tópico https://docs.djangoproject.com/en/2.2/ref/databases/.)
- ***name***: Define qual instância do banco de dados será acessada. Pense nesse parâmetro como o nome do banco que sua aplicação usará (por exemplo, se você está construindo um sistema de vendas, a instância poderia se chamar "db_vendas", ou algo parecido). Para o SQLite, ***sempre*** passe o caminho completo do arquivo do banco de dados, usando o caractere de barra (/) para especificar pastas, mesmo no Windows (por exemplo: /apps/consulta/db.sqlite3).

TABELA 14.1: Engines de bancos de dados

SGBD	engine
MySQL	django.db.backends.mysql
Oracle	django.db.backends.oracle
PostgreSQL	django.db.backends.postgresql
SQLite	django.db.backends.sqlite3

Além de ENGINE e NAME, existem outros parâmetros que podem ser ajustados e não vêm especificados por padrão em settings.py. A Tabela 14.2 mostra quais são eles. Se você quiser ajustar esses valores, lembre-se de que a configuração é um simples dicionário Python, então, basta adicionar linhas para os valores desejados (preste atenção: se, por descuido, copiar e colar um valor e esquecer de alterar a chave após a colagem, apenas um parâmetro será mudado e ficará com o valor que foi definido por último).

TABELA 14.2: Parâmetros de conexão

Parâmetro	Descrição
ATOMIC_REQUESTS	Define se cada requisição que partir de uma view será atômica ou não. Se for True, cada requisição fará parte de uma transação no SGBD e sempre será executada ou cancelada, não podendo existir execuções parciais. Seu valor padrão é False.
AUTOCOMMIT	Especifica se as transações contra o SGBD devem sofrer commit[4] automático. O padrão é True.
CONN_MAX_AGE	O tempo, em segundos, que uma conexão ao SGBD dura. O padrão é 0, significando que, a cada requisição, a conexão é encerrada após a operação. Para manter a conexão aberta permanentemente, passe o valor None (essa opção é desencorajada por alocar recursos desnecessariamente).
HOST	Especifica nome do host (computador) que contém o banco de dados. Uma string vazia (padrão) significa a máquina local.
OPTIONS	Um dicionário contendo opções dependentes do driver de BD usado. O valor padrão é um dicionário vazio ({ }).

Epílogo 263
Empacotando uma Aplicação com AMQP e JSP 279
O Django Framework 309
Modelos 337
Formulários e Templates 371
Criando Telas de Cadastro com o Django Admin 377
Construindo a Aplicação de Loja Virtual com o Django 425
Apêndice 1: Instalação de Ferramentas no Windows 430
Apêndice 2: Instalação de Ferramentas no Linux 433
Referências Bibliográficas

ATOMIC_REQUESTS	Define se cada requisição que partir de uma view será atômica ou não. Se for True, cada requisição fará parte de uma transação no SGBD e sempre será executada ou cancelada, não podendo existir execuções parciais. Seu valor padrão é False.
AUTOCOMMIT	Especifica se as transações contra o SGBD devem sofrer commit[8] automático. O padrão é True.
CONN_MAX_AGE	O tempo, em segundos, que uma conexão ao SGBD dura. O padrão é 0, significando que, a cada requisição, a conexão é encerrada após a operação. Para manter a conexão aberta permanentemente, passe o valor None (essa opção é desencorajada por alocar recursos desnecessariamente).
HOST	Especifica nome do host (computador) que contém o banco de dados. Uma string vazia (padrão) significa a máquina local.
OPTIONS	Um dicionário contendo opções dependentes do driver de BD usado. O valor padrão é um dicionário vazio ({ }).
PASSWORD	Senha para conexão ao SGBD. Não é utilizada para o SQLite.
PORT	Porta para conexão ao SGBD. Não é utilizada para o SQLite. Uma string vazia significa a porta default do driver utilizado.
USER	Usuário para conexão ao SGBD. Não é utilizada para o SQLite.

Instalando a biblioteca de manipulação de imagens Pillow

Um dos recursos de que a aplicação necessitará será a manipulação de imagens dos produtos. Para lidar com esse aspecto do projeto, aproveitarei para mostrar o uso de uma biblioteca externa. Instalarei a biblioteca ***Pillow***, muito usada para esse tipo de tarefa. No seu prompt de comandos/powershell/terminal, digite:

```
pip install pillow
```

Será exibido percentual de instalação. Quando esse chegar a 100%, aparecerá a mensagem:

```
Successfully installed pillow-<versão>
```

Instalando os pacotes de acesso a banco de dados do Python

Se você utilizará um SGBD diferente do SQLite, precisará instalar, também, o pacote de acesso a banco de dados para o SGBD escolhido. Para instalá-los, a forma mais prática é usar o PIP, o instalador de pacotes do Python. Se não o tiver em sua máquina, consulte os apêndices sobre instruções para obtê-lo. A Tabela 14.3 mostra como instalar no Django os drivers dos três SGBDs mais populares. O SQLite, como já explicado, vem instalado por padrão.

TABELA 14.3: Instalação dos drivers para diversos SGBDs

SGBD	comando
MySQL	pip install mysql-connector-python
Oracle	pip install cx_Oracle
PostgreSQL	pip install psycopg2-binary

AVISO

Podem ocorrer erros relacionados à ausência de bibliotecas de desenvolvimento para bancos de dados, sobretudo em sistemas baseados em Unix, como o Linux ou o MacOS. Essas bibliotecas não fazem parte do Python ou do Django, então, se acontecer algum problema na instalação, você precisará consultar a documentação do seu sistema operacional para corrigir o erro.

Testei todos os comandos da Tabela 14.3 em uma máquina com Ubuntu 14.04 LTS e eles funcionaram corretamente.

Expandir tudo | Recolher tudo

Estudo 285
Empacotando uma Aplicação com AMQP e RP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

Podem ocorrer erros relacionados à ausência de bibliotecas de desenvolvimento para bancos de dados, sobretudo em sistemas baseados em Unix, como o Linux ou o MacOS. Essas bibliotecas não fazem parte do Python ou do Django, então, se acontecer algum problema na instalação, você precisará consultar a documentação do seu sistema operacional para corrigir o erro.

Testei todos os comandos da Tabela 14.3 em uma máquina com Ubuntu 14.04 LTS e eles funcionaram corretamente.

Testando a conexão ao banco de dados

Agora que você configurou a conexão a um SGBD, está na hora de testar se ela funciona a contento. Abra um terminal/prompt de comandos e, dentro da pasta da sua aplicação (prj_lojavirtual, se você a renomeou como sugeri) e digite:

```
./manage.py migrate
```

Esse comando faz com que o Python crie várias tabelas administrativas no banco de dados da sua aplicação. Se tudo estiver configurado corretamente, você deverá ver uma sequência de mensagens semelhante à da Figura 14.6. Perceba que o Django cria para você diversas tabelas no banco de dados. Elas são necessárias para suportar, internamente, as funcionalidades da aplicação.

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
```

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

263
279
309
337
371
377
425
430
433

```
Applying contenttypes.0002_remove_content_type_name... OK
Applying auth.0002_alter_permission_name_max_length... OK
Applying auth.0003_alter_user_email_max_length... OK
Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK
Applying auth.0006_require_contenttypes_0002... OK
Applying auth.0007_alter_validators_add_error_messages... OK
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying sessions.0001_initial... OK
```

FIGURA 14.6: Mensagens confirmando criação de tabelas no banco de dados

DICA

As causas mais comuns de erros de conexão a bancos de dados são o fornecimento de credenciais (login e senha) incorretas, servidor de banco de dados não iniciado e ausência de algum driver de banco de dados no sistema operacional. Se houver qualquer erro nessa etapa, tente conectar-se ao seu SGBD com os mesmos parâmetros que usou no settings.py e verifique se o erro acontece — em muitos casos, é um simples erro de digitação.

Verificando o conteúdo de variáveis de ambiente do Django

O Django possui algumas variáveis de ambiente, que podem ser usadas para conferir informações sobre o seu projeto. Por exemplo, a variável BASE_DIR aponta para a pasta raiz de sua aplicação. Para conferir o conteúdo de alguma dessas variáveis, você pode abrir um shell de comandos a partir do seu projeto Django. Para isso, no terminal do Linux / prompt de comandos do Windows, acesse a pasta raiz do projeto (BASE_DIR) e digite:

```
./manage.py shell
```

Você verá alguns avisos, que variam de acordo com a sua instalação do Python, e o terminal passará a esperar comandos da linguagem Python, com a vantagem de ter todo o ambiente do seu projeto Django pré-carregado. Por exemplo, digite:

```
from django.conf import settings
```

Isso importará o arquivo settings.py do pacote django.conf, que contém as configurações do seu projeto. Agora, para conferir qual é a pasta raiz do projeto, digite:

```
print(settings.BASE_DIR)
```

que imprimirá o valor da constante BASE_DIR, do arquivo settings.py do projeto atual. Para sair do shell do Python, digite:

```
exit()
```

Expandir tudo | Recolher tudo

Resumo 255
Preparando uma Aplicação com AMDG & JQ 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação da Loja Virtual com o Django 377
Apêndice 1: Instalação de Ferramentas no Windows 425
Apêndice 2: Instalação de Ferramentas no Linux 430
Referências Bibliográficas 433

```
print(settings.BASE_DIR)
```

que imprimirá o valor da constante BASE_DIR, do arquivo settings.py do projeto atual. Para sair do shell do Python, digite:

```
exit()
```

Personalizando o urls.py

O arquivo urls.py é o local padrão em que você pode definir as URLs de seu projeto Django. Lá você configura toda a mágica que faz uma requisição de determinada página da sua aplicação ser direcionada a uma ***view***, que, por sua vez, chamará os métodos da camada de negócios que atenderão ao pedido.

A lista urlpatterns define um mapeamento entre URLs internas à sua aplicação e os métodos/funções de suas views, de maneira que, se uma requisição for realizada para um endereço que "case" com o padrão da URL, ela será redirecionada para view apropriada, que irá tratá-la. É sua responsabilidade, como desenvolvedor, decidir qual view deverá responder a cada requisição gerada pelos cliques na interface com o usuário da aplicação. Isso é feito por meio da definição de uma configuração de URL. Como urlpatterns é uma lista do Python, adicionar um novo mapeamento é uma questão de, simplesmente, digitar um novo item na lista. Cada item é mapeado pela função path(), que recebe dois parâmetros:

- A ***URL*** do mapeamento. Ou seja, o que deverá ser digitado após o endereço do seu projeto para chamar a view a ser mapeada.
- O identificador do recurso a ser mapeado: ***pacote.classe*** ou ***função.propriedade*** ou ***método***.

Digamos que você queira mapear um link cujo endereço dentro do seu site seja seuapp:8000/meulink para que ele chame o método criar_algo() de uma view denominada view_criadora, contida em um pacote chamado viewspublicas. Você precisaria adicionar a linha

```
path('meulink/', viewspublicas.view_criadora.criar_algo),
```

O mapeamento da Listagem 14.9 significa que, se o usuário da sua aplicação digitar no browser o endereço do seu projeto Django, seguido de admin/, será chamada a view correspondente à página de administração do site.

O mecanismo funciona assim: o arquivo urls.py declara quais URLs estão disponíveis e quais métodos/funções das views devem responder por elas. Quando uma solicitação a uma dessas URLs chega à aplicação, o Django confere se existe um mapeamento para ela e a encaminha para a view adequada, definida no arquivo views.py.

A view, então, executa o código do método/função associado à URL e devolve uma página (***template***) com a resposta para o usuário.

Quando você cria um novo projeto, o conteúdo desse arquivo é apenas o da Listagem 14.9 (nota: omiti os comentários no cabeçalho do arquivo).

```
from django.contrib import admin

from django.urls import path

urlpatterns = [

path('admin/', admin.site.urls),

]
```

LISTAGEM 14.9: Conteúdo inicial do arquivo urls.py

O que nos interessa é a chamada à função path(), que mapeia as URLs internas para métodos/funções das views no projeto. No exemplo da Listagem 14.9:

- admin/ - URL: Significa que, se o usuário digitar o endereço da sua aplicação, digamos, http://127.0.0.1:8000/ seguido de admin, ou seja, http://127.0.0.1:8000/admin, a requisição será redirecionada para o recurso listado no segundo parâmetro da função.
- admin.site.urls: Esta classe confunde ***bastante*** os iniciantes. Se você quiser ver o código dela, precisará ir até a pasta ***site-packages/django/contrib/admin/*** da sua instalação do Python e abrir o arquivo ***__init__.py*** (aquele que é usado para marcar a pasta como um pacote Python, como citei no Capítulo 9, na seção "Agrupando módulos em pacotes"). Nele, você encontrará uma linha: ***from django.contrib.admin.sites import AdminSite, site***. Olhe agora o conteúdo de ***django/contrib/admin/sites.py***. Na última linha do arquivo, há uma variável global com o valor ***site = DefaultAdminSite()***. Lembre-se de que o interpretador buscará pelo

Expandir tudo | Recolher tudo

Estado 255
Empacotando uma Aplicação com AMQP e JSP 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Aplicação de Leilão Virtual com o Django 377
Apêndice 1: Instalando as Ferramentas no Windows 425
Apêndice 2: Instalando as Ferramentas no Linux 430
Referências Bibliográficas 433

listado no segundo parâmetro da função.

- admin.site.urls: Esta classe confunde ***bastante*** os iniciantes. Se você quiser ver o código dela, precisará ir até a pasta ***site-packages/django/contrib/admin/*** da sua instalação do Python e abrir o arquivo ***__init__.py*** (aquele que é usado para marcar a pasta como um pacote Python, como citei no Capítulo 9, na seção "Agrupando módulos em pacotes"). Nele, você encontrará uma linha: ***from django.contrib.admin.sites import AdminSite, site***. Olhe agora o conteúdo de ***django/contrib/admin/sites.py***. Na última linha do arquivo, há uma variável global com o valor ***site = DefaultAdminSite()***. Lembre-se de que o interpretador buscará pelo código usando o ***caminho de pesquisa*** (***serch path***), conforme explicado na seção "Como o Python encontra seus módulos", no Capítulo 9. Ou seja, se a pasta site-packages da sua máquina estiver em c:\python\site-packages, o arquivo estará em ***c:\python\site-packages\django\contrib\admin\sites.py***. Em resumo, essa linha instrui o interpretador a redirecionar qualquer chamada com a url http://***<Endereço do seu servidor>***:***<Porta usada pelo Python>***/admin para o valor retornado pela propriedade ***urls*** de um objeto da classe ***DefaultAdminSite***.

É possível, ainda, passar parâmetros para as views mapeadas: eles devem ser especificados entre ***chaves angulares*** (<>) e terem seu tipo declarado no formato: ***<tipo:nome_do_parâmetro>***. Por exemplo, se você quiser que qualquer URL terminada em get/, seguida por um número inteiro que será passado em um parâmetro de nome ***id***, e outra barra a um método chamado ***meu_metodo()***, declarado em uma view de nome ***minhaclasse***, que faz parte de um módulo chamado ***modulo***, use:

```
path('get/<int:id>/', modulo.minhaclasse.metodo)
```

A Tabela 14.4 mostra como converter um mapeamento no formato antigo, com a função url(), para o novo formato, que usa a função path().

DICA

Relendo esse último tópico, sei que pareceu muito "enrolado" para os iniciantes, mas abstraia isso, por ora. Pense nessa linha como "se alguém digitar ***<endereço do meu servidor:8000>/admin***, ele redirecionará a chamada para uma *view* que exibe a aplicação de administração do Django". Essa aplicação será vista mais detalhadamente no Capítulo 17, quando eu criarei o programa exemplo do livro.

AVISO

Antes da versão 2.0 do Django, o mapeamento de URLs no arquivo urls.py era realizado por outra função: url(), que utilizava ***expressões regulares***[1] para especificar os padrões de URLs que serão mapeados para métodos/funções das views. Você ainda pode fazê-lo, porém o novo formato é bem mais amigável.

Exemplo de mapeamento realizado com a função url():

```
url(r'^lances/(?P<lance_id>[0-9]+)/$', view_detalhes_lance)
```

O mesmo mapeamento efetuado com a função path():

```
path('lances/<int:lance_id>/', view_detalhes_lance)
```

TABELA 14.4: Equivalência entre formatos de parâmetros com as funções url() e path()

Exemplos de conversão de formatos entre a função url() e a função path()	
Usando *url()*	**Usando** *path()*
[0-9]+	int
[^/]+	str
[-a-zA-Z0-9_]+	slug
.*	path

[1] ***REST (REpresentational State Transfer)*** é um padrão arquitetural para ***web services*** que vem ganhando força há alguns anos. Nele, são usados os ***verbos do protocolo HTTP*** (GET, PUT, POST e DELETE) para chamar operações de um serviço disponibilizado por uma dada aplicação de modo

Expandir tudo | Recolher tudo

263
279
309
337
371
377
425
430
433

de um módulo chamado ***modulo***, use:

```
path('get/<int:id>/', modulo.minhaclasse.metodo)
```

A Tabela 14.4 mostra como converter um mapeamento no formato antigo, com a função url(), para o novo formato, que usa a função path().

DICA

Relendo esse último tópico, sei que pareceu muito "enrolado" para os iniciantes, mas abstraia isso, por ora. Pense nessa linha como "se alguém digitar *<endereço do meu servidor:8000>/admin*, ele redirecionará a chamada para uma *view* que exibe a aplicação de administração do Django". Essa aplicação será vista mais detalhadamente no Capítulo 17, quando eu criarei o programa exemplo do livro.

AVISO

Antes da versão 2.0 do Django, o mapeamento de URLs no arquivo urls.py era realizado por outra função: url(), que utilizava ***expressões regulares***[8] para especificar os padrões de URLs que serão mapeados para métodos/funções das views. Você ainda pode fazê-lo, porém o novo formato é bem mais amigável.

Exemplo de mapeamento realizado com a função url():

```
url(r'^lances/(?P<lance_id>[0-9]+)/$', view_detalhes_lance)
```

O mesmo mapeamento efetuado com a função path():

```
path('lances/<int:lance_id>/', view_detalhes_lance)
```

TABELA 14.4: Equivalência entre formatos de parâmetros com as funções url() e path()

Exemplos de conversão de formatos entre a função url() e a função path()	
Usando url()	**Usando** path()
[0-9]+	int
[^/]+	str
[-a-zA-Z0-9_]+	slug
.*	path

[1] ***REST (REpresentational State Transfer)*** é um padrão arquitetural para ***web services*** que vem ganhando força há alguns anos. Nele, são usados os ***verbos do protocolo HTTP*** (GET, PUT, POST e DELETE) para chamar operações de um serviço disponibilizado por uma dada aplicação de modo simplificado.

[2] Em um dos livros que consta das referências bibliográficas, *Djangoweb/e Django*, os autores fizeram exatamente isso: criaram um projeto no estilo "*Hello, World!*" em Django, cujo código está todo completo em um único arquivo. Apenas para fins didáticos, não existe problema, mas, em projetos reais, esteja avisado(a): não faça isso.

[3] URL é a sigla de ***Uniform Resource Locator*** — Localizador de Recurso Uniforme. Entenda-o como um endereço da web, que você digita no seu navegador quando quer acessar uma página. Por exemplo: http://www.altabooks.com.br é uma URL, assim como 127.0.0.1:8000.

[4] Propositalmente, usei exemplos de endereços que não existem. Embora alguém possa registrar o endereço www.qualquercoisa.com, o endereço 192.168.1.250 é *reservado* e não pode ser usado fora de uma rede local.

[5] O termo *logging* refere-se ao "ato de registrar alguma informação no log", tipicamente algum evento do seu projeto, como um erro ocorrido ou uma mensagem informativa. *Log*, por sua vez, é a própria listagem dessas informações.

[6] ***ODBC (Open DataBase Connectivity)*** é uma API que permite conectar aplicações com diversos bancos de dados, desde que haja um driver ODBC compatível para utilizá-la com o Django.

[7] Lembre-se da seção "Usando dicionários para criar tabelas", do Capítulo 7, em que você aprendeu que o tipo dicionário pode conter outros dicionários ou valores simples — esse conhecimento, agora, o(a) ajudará a compreender melhor a estrutura desse arquivo.

[8] Commit, em bancos de dados, é a operação que confirma a execução de uma transação. Após ser emitida, a operação de manipulação de dados que ela confirmou não pode ser mais desfeita.

[9] ***Expressões regulares*** são padrões de caracteres que associam strings em um texto. São muito usadas em diversas áreas da computação.